SOFTWARE ENGINEERING PROJECT MANAGEMENT – BCS501

PREPARED BY
PRAKRUTHI D P
ASSISTANT PROFESSOR
BIT

MODULE-2

REQUIREMENTS ENGINEERING:

- **UNDERSTANDING REQUIREMENTS:** REQUIREMENTS ENGINEERING, ESTABLISHING THE GROUND WORK, ELICITING REQUIREMENTS, DEVELOPING USE CASES, BUILDING THE REQUIREMENTS MODEL, NEGOTIATING REQUIREMENTS, VALIDATING REQUIREMENTS.
- **REQUIREMENTS MODELING SCENARIOS, INFORMATION AND ANALYSIS CLASSES:** REQUIREMENT ANALYSIS, SCENARIO BASED MODELING, UML MODELS THAT SUPPLEMENT THE USE CASE, DATA MODELING CONCEPTS, CLASS-BASED MODELING.
- **REQUIREMENT MODELING STRATEGIES** : FLOW ORIENTED MODELING , BEHAVIORAL MODELING.

REQUIREMENT ENGINEERING

- THE PROCESS OF COLLECTING THE SOFTWARE REQUIREMENT FROM THE CLIENT THEN UNDERSTAND, EVALUATE AND DOCUMENT IT IS CALLED AS **REQUIREMENT ENGINEERING.**
- REQUIREMENT ENGINEERING CONSTRUCTS A BRIDGE FOR DESIGN AND CONSTRUCTION.

REQUIREMENT ENGINEERING TASKS

- INCEPTION
- ELICITATION
- ELABORATION
- NEGOTIATION
- SPECIFICATION
- VALIDATION
- REQUIREMENT MANAGEMENT

REQUIREMENT ENGINEERING TASKS

- **INCEPTION**

AT PROJECT INCEPTION, **YOU ESTABLISH A BASIC UNDERSTANDING OF THE PROBLEM, THE PEOPLE WHO WANT A SOLUTION, THE NATURE OF THE SOLUTION THAT IS DESIRED,** AND THE EFFECTIVENESS OF PRELIMINARY COMMUNICATION AND COLLABORATION BETWEEN THE OTHER STAKEHOLDERS AND THE SOFTWARE TEAM.

- **ELICITATION**

IT CERTAINLY SEEMS SIMPLE ENOUGH—ASK THE CUSTOMER, THE USERS, AND OTHERS WHAT THE OBJECTIVES FOR THE SYSTEM OR PRODUCT ARE, WHAT IS TO BE ACCOMPLISHED, HOW THE SYSTEM OR PRODUCT FITS INTO THE NEEDS OF THE BUSINESS, AND FINALLY, HOW THE SYSTEM OR PRODUCT IS TO BE USED ON A DAY-TO-DAY BASIS.

REQUIREMENT ENGINEERING TASKS

ELABORATION

THE INFORMATION OBTAINED FROM THE **CUSTOMER DURING INCEPTION AND ELICITATION IS EXPANDED AND REFINED DURING ELABORATION**. THIS TASK FOCUSES ON DEVELOPING A REFINED REQUIREMENTS MODEL THAT IDENTIFIES VARIOUS ASPECTS OF SOFTWARE FUNCTION, BEHAVIOR, AND INFORMATION.

NEGOTIATION. IT ISN'T UNUSUAL FOR CUSTOMERS AND USERS TO ASK FOR MORE THAN CAN BE ACHIEVED, GIVEN LIMITED BUSINESS RESOURCES. IT'S ALSO RELATIVELY COMMON FOR DIFFERENT CUSTOMERS OR USERS TO PROPOSE CONFLICTING REQUIREMENTS, ARGUING THAT THEIR VERSION IS "ESSENTIAL FOR OUR SPECIAL NEEDS."

REQUIREMENT ENGINEERING TASKS

- **Specification**. In the context of computer-based systems (and software), the term **specification means different things to different people**. A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.
- **Validation**. The work products produced as a **consequence of requirements** engineering are assessed for quality during a validation step. Requirements validation examines the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work products conform to the standards established for the process, the project, and the product.

REQUIREMENT ENGINEERING TASKS

REQUIREMENTS MANAGEMENT. REQUIREMENTS FOR **COMPUTER-BASED SYSTEMS CHANGE, AND THE DESIRE TO CHANGE REQUIREMENTS** PERSISTS THROUGHOUT THE LIFE OF THE SYSTEM. REQUIREMENTS MANAGEMENT IS A SET OF ACTIVITIES THAT HELP THE PROJECT TEAM IDENTIFY, CONTROL, AND TRACK REQUIREMENTS AND CHANGES TO REQUIREMENTS AT ANY TIME AS THE PROJECT PROCEEDS.

THE 4 STEPS OF THE REQUIREMENTS ENGINEERING PROCESS

- Eliciting requirements.
- Requirements specification(FR & NFR)
- Verification and validation.
- Requirements management.

ESTABLISHING THE GROUND WORK SEPM - BCS501

- In an ideal setting, **stakeholders** and **software engineers** work together on the same team. In such cases, requirements engineering is simply a matter of conducting meaningful conversations with colleagues who are well-known members of the team. But reality is often quite different.
- **Customer(s)** or end users may be located in a different city or country, may have only a vague idea of what is required, may have conflicting opinions about the system to be built, may have limited technical knowledge, and may have limited time to interact with the requirements engineer. None of these things are desirable, but all are fairly common, and you are often forced to work within the constraints imposed by this situation.

ESTABLISHING THE GROUND WORK

- Identifying Stakeholders
- Recognizing Multiple Viewpoints
- Working toward Collaboration
- Asking the First Questions

ESTABLISHING THE GROUND WORK

- **IDENTIFYING STAKEHOLDERS**

EACH STAKEHOLDER HAS A DIFFERENT VIEW OF THE SYSTEM, ACHIEVES DIFFERENT BENEFITS WHEN THE SYSTEM IS SUCCESSFULLY DEVELOPED, AND IS OPEN TO DIFFERENT RISKS IF THE DEVELOPMENT EFFORT SHOULD FAIL.

- **RECOGNIZING MULTIPLE VIEWPOINTS**

BUSINESS MANAGERS ARE INTERESTED IN A FEATURE SET THAT CAN BE BUILT WITHIN BUDGET AND THAT WILL BE READY TO MEET DEFINED MARKET WINDOWS. END USERS MAY WANT FEATURES THAT ARE FAMILIAR TO THEM AND THAT ARE EASY TO LEARN AND USE. SOFTWARE ENGINEERS MAY BE CONCERNED WITH FUNCTIONS THAT ARE INVISIBLE TO NONTECHNICAL STAKEHOLDERS BUT THAT ENABLE AN INFRASTRUCTURE THAT SUPPORTS MORE MARKETABLE FUNCTIONS AND FEATURES. SUPPORT ENGINEERS MAY FOCUS ON THE MAINTAINABILITY OF THE SOFTWARE.

ESTABLISHING THE GROUND WORK

- **WORKING TOWARD COLLABORATION**

COLLABORATION DOES NOT NECESSARILY MEAN THAT REQUIREMENTS ARE DEFINED BY COMMITTEE. IN MANY CASES, STAKEHOLDERS COLLABORATE BY PROVIDING THEIR VIEW OF REQUIREMENTS, BUT A STRONG **"PROJECT CHAMPION" (**E.G., A BUSINESS MANAGER OR A SENIOR TECHNOLOGIST) MAY MAKE THE FINAL DECISION ABOUT WHICH REQUIREMENTS MAKE THE CUT.

- **ASKING THE FIRST QUESTIONS**

QUESTIONS ASKED AT THE INCEPTION OF THE PROJECT SHOULD BE "CONTEXT FREE" [GAU89]. THE FIRST SET OF CONTEXT-FREE QUESTIONS FOCUSES ON THE CUSTOMER AND OTHER STAKEHOLDERS, THE OVERALL PROJECT GOALS AND BENEFITS.

Who is behind the request for this work?

- Who will use the solution?
- What will be the economic benefit of a successful solution?
- Is there another source for the solution that you need?
- These questions help to identify all stakeholders who will have interest in the software to be built. In addition, the questions identify the measurable benefit of a successful implementation and possible alternatives to custom software development.

ELICITING REQUIREMENTS

- **REQUIREMENTS ELICITATION** (ALSO CALLED REQUIREMENTS GATHERING) COMBINES ELEMENTS OF PROBLEM SOLVING, ELABORATION, NEGOTIATION, AND SPECIFICATION.
- IN ORDER TO ENCOURAGE A **COLLABORATIVE, TEAM-ORIENTED** APPROACH TO **REQUIREMENTS GATHERING**, **STAKEHOLDERS** WORK TOGETHER TO IDENTIFY THE PROBLEM, PROPOSE ELEMENTS OF THE SOLUTION, NEGOTIATE DIFFERENT APPROACHES AND SPECIFY A PRELIMINARY SET OF SOLUTION REQUIREMENTS [ZAH90].

DIFFERENT APPROACHES FOR REQUIREMENT ELICITATION

- Collaborative Requirements Gathering
- Quality Function Deployment
- Usage Scenarios
- Elicitation Work Products

ELICITING REQUIREMENTS

- **COLLABORATIVE REQUIREMENTS GATHERING**
- MANY DIFFERENT APPROACHES TO COLLABORATIVE REQUIREMENTS GATHERING HAVE BEEN PROPOSED. EACH MAKES USE OF A SLIGHTLY DIFFERENT SCENARIO, BUT ALL APPLY SOME VARIATION ON THE FOLLOWING BASIC GUIDELINES:
- MEETINGS ARE CONDUCTED AND ATTENDED BY BOTH SOFTWARE ENGINEERS AND OTHER STAKEHOLDERS.
- RULES FOR PREPARATION AND PARTICIPATION ARE ESTABLISHED.
- AN AGENDA IS SUGGESTED THAT IS FORMAL ENOUGH TO COVER ALL IMPORTANT POINTS BUT INFORMAL ENOUGH TO ENCOURAGE THE FREE FLOW OF IDEAS.

ELICITING REQUIREMENTS

- **QUALITY FUNCTION DEPLOYMENT**
- QUALITY FUNCTION DEPLOYMENT (QFD) IS A QUALITY MANAGEMENT TECHNIQUE THAT TRANSLATES THE NEEDS OF THE CUSTOMER INTO TECHNICAL REQUIREMENTS FOR SOFTWARE. QFD "**CONCENTRATES ON MAXIMIZING CUSTOMER SATISFACTION FROM THE SOFTWARE ENGINEERING PROCESS**"
- TO ACCOMPLISH THIS, QFD EMPHASIZES AN UNDERSTANDING OF WHAT IS VALUABLE TO THE CUSTOMER AND THEN DEPLOYS THESE VALUES THROUGHOUT THE ENGINEERING PROCESS.
- **QFD IDENTIFIES THREE TYPES OF REQUIREMENTS.**
- NORMAL REQUIREMENTS.
- EXPECTED REQUIREMENTS.
- EXCITING REQUIREMENTS

QUALITY FUNCTION DEPLOYMENT

- **Normal requirements**
- **Example** of normal requirements might be requested types of graphical displays, specific system functions, and defined levels of performance.
- **Expected requirements**
- **Examples** of expected requirements are: ease of human/machine interaction overall operational correctness and reliability, and ease of software installation.
- **Exciting requirements**
- **For example**, software for a new mobile phone comes with standard features, but is coupled with a set of unexpected capabilities (e.g., multitouch screen, visual voice mail) that delight every user of the product

ELICITING REQUIREMENTS

- **USAGE SCENARIOS**
- AN OVERALL VISION OF SYSTEM **FUNCTIONS AND FEATURES** BEGINS TO MATERIALIZE. HOWEVER, IT IS DIFFICULT TO MOVE INTO MORE TECHNICAL SOFTWARE ENGINEERING ACTIVITIES UNTIL YOU UNDERSTAND FUNCTIONS AND FEATURES WILL BE USED BY DIFFERENT CLASSES OF END USERS.
- DEVELOPERS AND USERS CAN CREATE A SET OF SCENARIOS THAT IDENTIFY A THREAD OF USAGE FOR THE SYSTEM TO BE CONSTRUCTED. THE SCENARIOS, OFTEN CALLED **USE CASES** [JAC92]

ELICITING REQUIREMENTS

SEPM - BCS501

ELICITATION WORK PRODUCTS

THE WORK PRODUCTS PRODUCED AS A CONSEQUENCE OF REQUIREMENTS ELICITATION WILL VARY DEPENDING ON THE SIZE OF THE SYSTEM OR PRODUCT TO BE BUILT. FOR MOST SYSTEMS, THE WORK PRODUCTS INCLUDE

- A STATEMENT OF NEED AND FEASIBILITY.
- A BOUNDED STATEMENT OF SCOPE FOR THE SYSTEM OR PRODUCT.
- A LIST OF CUSTOMERS, USERS, AND OTHER STAKEHOLDERS WHO PARTICIPATED IN REQUIREMENTS ELICITATION.
- A DESCRIPTION OF THE SYSTEM'S TECHNICAL ENVIRONMENT.
- A LIST OF REQUIREMENTS (PREFERABLY ORGANIZED BY FUNCTION) AND THE DOMAIN CONSTRAINTS THAT APPLY TO EACH.
- A SET OF USAGE SCENARIOS THAT PROVIDE INSIGHT INTO THE USE OF THE SYSTEM OR PRODUCT UNDER DIFFERENT OPERATING CONDITIONS.
- ANY PROTOTYPES DEVELOPED TO BETTER DEFINE REQUIREMENTS

ELICITING REQUIREMENTS

- **STAKEHOLDER INTERVIEWS:** DISCUSSING NEEDS, EXPECTATIONS, AND PAIN POINTS WITH END-USERS, DOMAIN EXPERTS, AND PROJECT MANAGERS.
- **WORKSHOPS AND BRAINSTORMING SESSIONS:** FACILITATING GROUP DISCUSSIONS TO UNCOVER IDEAS AND IDENTIFY FUNCTIONALITIES.
- **DOCUMENT ANALYSIS:** REVIEWING EXISTING DOCUMENTS LIKE BUSINESS PROCESS MODELS OR USER MANUALS.
- **SURVEYS AND QUESTIONNAIRES:** GATHERING QUANTITATIVE DATA ON USER PREFERENCES AND PRIORITIES.
- **PROTOTYPING:** BUILDING BASIC MODELS TO GAIN USER FEEDBACK AND REFINE REQUIREMENTS.

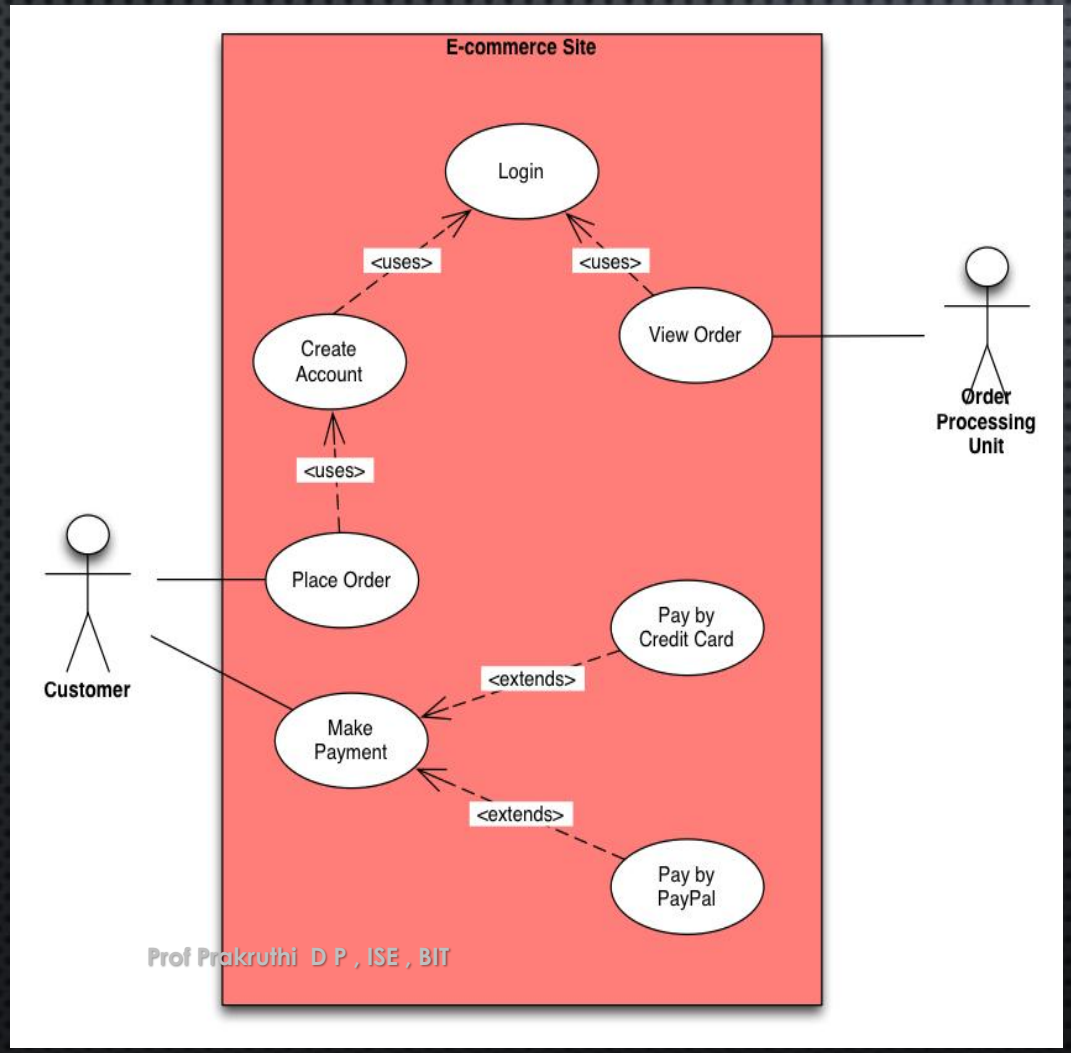
DEVELOPING USE-CASES

- A use case diagram is a visual representation of how users interact with a system to achieve specific goals. It's a core part of Unified Modeling Language (UML) and is used in software design and development.

COMPONENTS OF USE CASE DIAGRAM

- **ACTORS:** THESE REPRESENT ANYTHING THAT INTERACTS WITH THE SYSTEM, LIKE A PERSON (CUSTOMER, ADMINISTRATOR), ANOTHER SYSTEM, OR AN EXTERNAL DEVICE. BY IDENTIFYING THE ACTORS, **YOU UNDERSTAND WHO USES THE SYSTEM AND FOR WHAT PURPOSES.**
- **USE CASES**: THESE ARE FUNCTIONALITIES OR FEATURES OFFERED BY THE SYSTEM. EACH USE CASE REPRESENTS A COMPLETE FLOW OF ACTIVITIES WHERE AN ACTOR INTERACTS WITH THE SYSTEM TO ACHIEVE A GOAL. **USE CASES HELP DEFINE THE SYSTEM'S FUNCTIONALITIES FROM THE USER'S PERSPECTIVE.**
- **RELATIONSHIPS**: USE CASE DIAGRAMS SHOW HOW ACTORS AND USE CASES CONNECT. THIS HELPS VISUALIZE HOW DIFFERENT FUNCTIONALITIES WORK TOGETHER AND HOW USERS INTERACT WITH THEM. THERE ARE DIFFERENT TYPES OF RELATIONSHIPS, LIKE **INCLUDES (COMMON FUNCTIONALITY) AND EXTENDS (OPTIONAL FUNCTIONALITY),** THAT PROVIDE A MORE NUANCED UNDERSTANDING OF THE SYSTEM'S BEHAVIOR.

IMPORTANCE OF USE CASE DIAGRAMS:



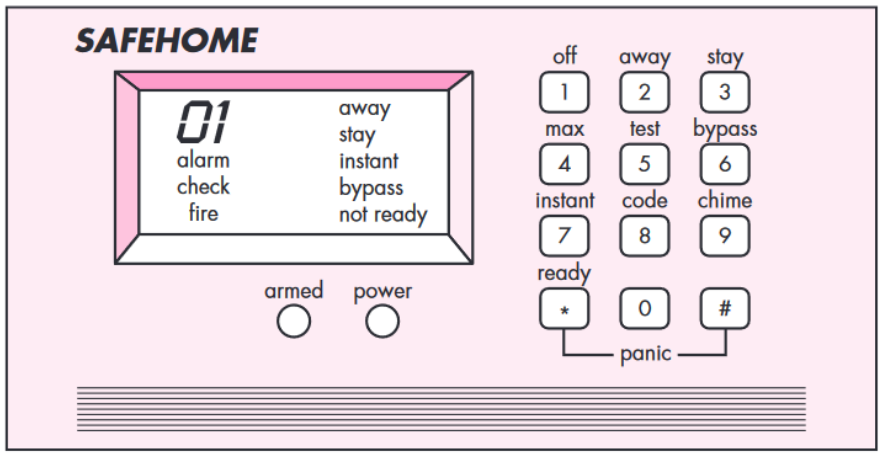
- **Functionality Depiction:**
- **Early Identification of Issues**
- **System Scope Definition**
- **Documentation**

- In essence, use case diagrams offer a high-level overview of a system's functionalities and user interactions. This visual representation fosters better communication, helps identify potential problems early on, and lays the foundation for successful system development.

SAFE HOME SYSTEM(PLEASE REFER TEXT BOOK FOR DETAILED UNDERSTANDING SAFE HOME)

FIGURE 5.1

SafeHome control panel



13 Note that this use case differs from the situation in which the system is accessed via the Internet. In this case, interaction occurs via the control panel, not the graphical user interface (GUI) provided when a PC is used.

SAFE HOME SYSTEM

- **OBJECTS** DESCRIBED FOR SAFE HOME MIGHT INCLUDE THE **CONTROL PANEL**,
- **SMOKE DETECTORS**, WINDOW AND DOOR SENSORS , MOTION DETECTORS,
- **AN ALARM**, AN EVENT (A SENSOR HAS BEEN ACTIVATED), A DISPLAY, A PC, TELEPHONE NUMBERS, A TELEPHONE CALL, AND SO ON.
- THE LIST OF SERVICES MIGHT INCLUDE **CONFIGURING THE SYSTEM**, **SETTING THE ALARM, MONITORING THE SENSORS**, DIALING THE PHONE,
- PROGRAMMING THE CONTROL PANEL, AND READING THE DISPLAY (NOTE THAT SERVICES ACT ON OBJECTS).
- IN A SIMILAR FASHION, EACH ATTENDEE WILL DEVELOP
- **LISTS OF CONSTRAINTS** (E.G., THE SYSTEM MUST RECOGNIZE WHEN SENSORS ARE NOT OPERATING, MUST BE USER-FRIENDLY, MUST INTERFACE DIRECTLY TO A STANDARD PHONE LINE) AND PERFORMANCE CRITERIA (E.G., A SENSOR EVENT SHOULD BE RECOGNIZED WITHIN ONE SECOND, AND AN EVENT PRIORITY SCHEME SHOULD BE IMPLEMENTED)

SAFE HOME SYSTEM

Recalling basic Safe Home requirements

- We define four actors: **homeowner**(a user),
- **Setup manager** (likely the same person as homeowner, but playing a different role),
- **Sensors** (devices attached to the system) and the monitoring and response subsystem (the central station that monitors the **Safe Home home security** function).
- For the purposes of this example, we consider only the **homeowner actor**. The homeowner actor interacts with the home security function in a number of different ways using either the alarm control panel or a PC:

SAFE HOME SYSTEM

- Enters a password to allow all **other interactions.**
- Inquires about the status of a **security zone.**
- Inquires about the status of a **sensor.**
- Presses the panic button in an **emergency.**
- Activates/deactivates the **security system.**

SAFE HOME SYSTEM

- **Considering the situation in which the homeowner uses the control panel, the basic use case for system activation follows:**
- 1. The homeowner observes the Safe Home control panel to determine if the system is ready for input. If the system is not ready, a not ready message is displayed on the LCD display, and the homeowner must physically close windows or doors so that the not ready message disappears. [A not ready message implies that a sensor is open; i.e., that a door or window is open.]

SAFE HOME SYSTEM

2.The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.

3. The homeowner selects and keys in stay or away (see Figure 5.1) to activate the system. Stay activates only perimeter sensors (inside motion detecting sensors are deactivated). Away activates all sensors.

4. When activation occurs, a red alarm light can be observed by the homeowner.

REQUIREMENTS SPECIFICATION

- **FUNCTIONAL REQUIREMENTS:** FUNCTIONAL REQUIREMENTS DEFINE A FUNCTION THAT A SYSTEM OR SYSTEM ELEMENT MUST BE QUALIFIED TO PERFORM AND MUST BE DOCUMENTED IN DIFFERENT FORMS. THE FUNCTIONAL REQUIREMENTS ARE DESCRIBING THE BEHAVIOR OF THE SYSTEM AS IT CORRELATES TO THE SYSTEM'S FUNCTIONALITY.
- **NON-FUNCTIONAL REQUIREMENTS:** THIS CAN BE THE NECESSITIES THAT SPECIFY THE CRITERIA THAT CAN BE USED TO DECIDE THE OPERATION INSTEAD OF SPECIFIC BEHAVIORS OF THE SYSTEM. NON-FUNCTIONAL REQUIREMENTS ARE DIVIDED INTO TWO MAIN CATEGORIES:

BUILDING THE REQUIREMENT MODELS

- THE INTENT OF THE ANALYSIS MODEL IS TO PROVIDE A DESCRIPTION OF THE REQUIRED **INFORMATIONAL, FUNCTIONAL, AND BEHAVIORAL DOMAINS FOR A COMPUTER-BASED SYSTEM.**
- THE MODEL CHANGES DYNAMICALLY AS YOU LEARN MORE ABOUT THE SYSTEM TO BE BUILT, AND OTHER **STAKEHOLDERS UNDERSTAND** MORE ABOUT WHAT THEY REALLY REQUIRE

BUILDING THE REQUIREMENT MODELS

- Scenario-based elements.
- Class-based elements.
- Behavioral elements.

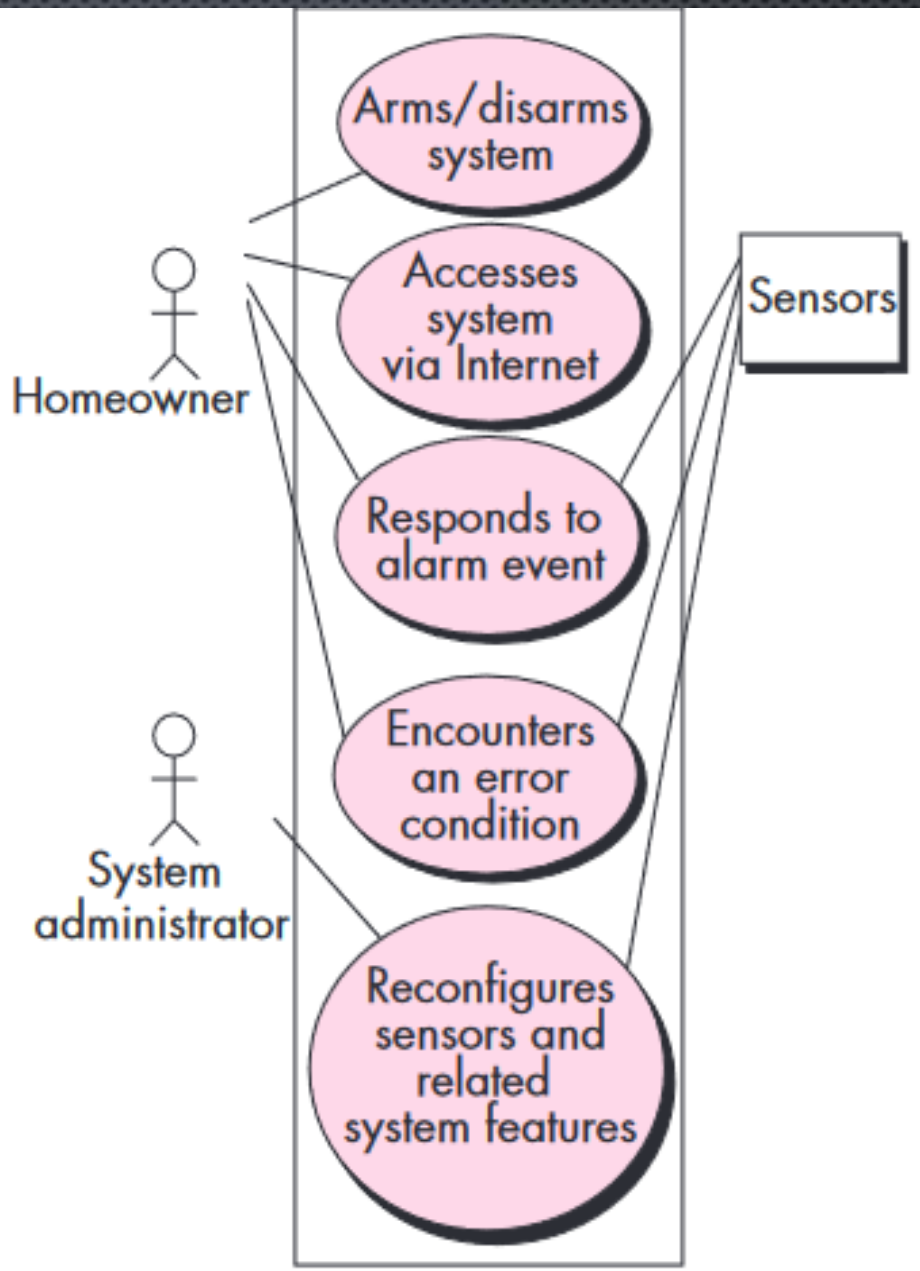
ELEMENTS OF THE REQUIREMENTS MODEL

- **SCENARIO-BASED ELEMENTS**

THE SYSTEM IS DESCRIBED FROM THE USER'S POINT OF VIEW USING A SCENARIO-BASED APPROACH. **FOR EXAMPLE**, BASIC USE CASES (SECTION 5.4) AND THEIR CORRESPONDING USE-CASE DIAGRAMS (FIGURE 5.2) EVOLVE INTO MORE ELABORATE TEMPLATE-BASED USE CASES.

FIGURE 5.3 DEPICTS A UML ACTIVITY DIAGRAM FOR ELICITING REQUIREMENTS AND REPRESENTING THEM USING USE CASES. THREE LEVELS OF ELABORATION ARE SHOWN, CULMINATING IN A SCENARIO-BASED REPRESENTATION.

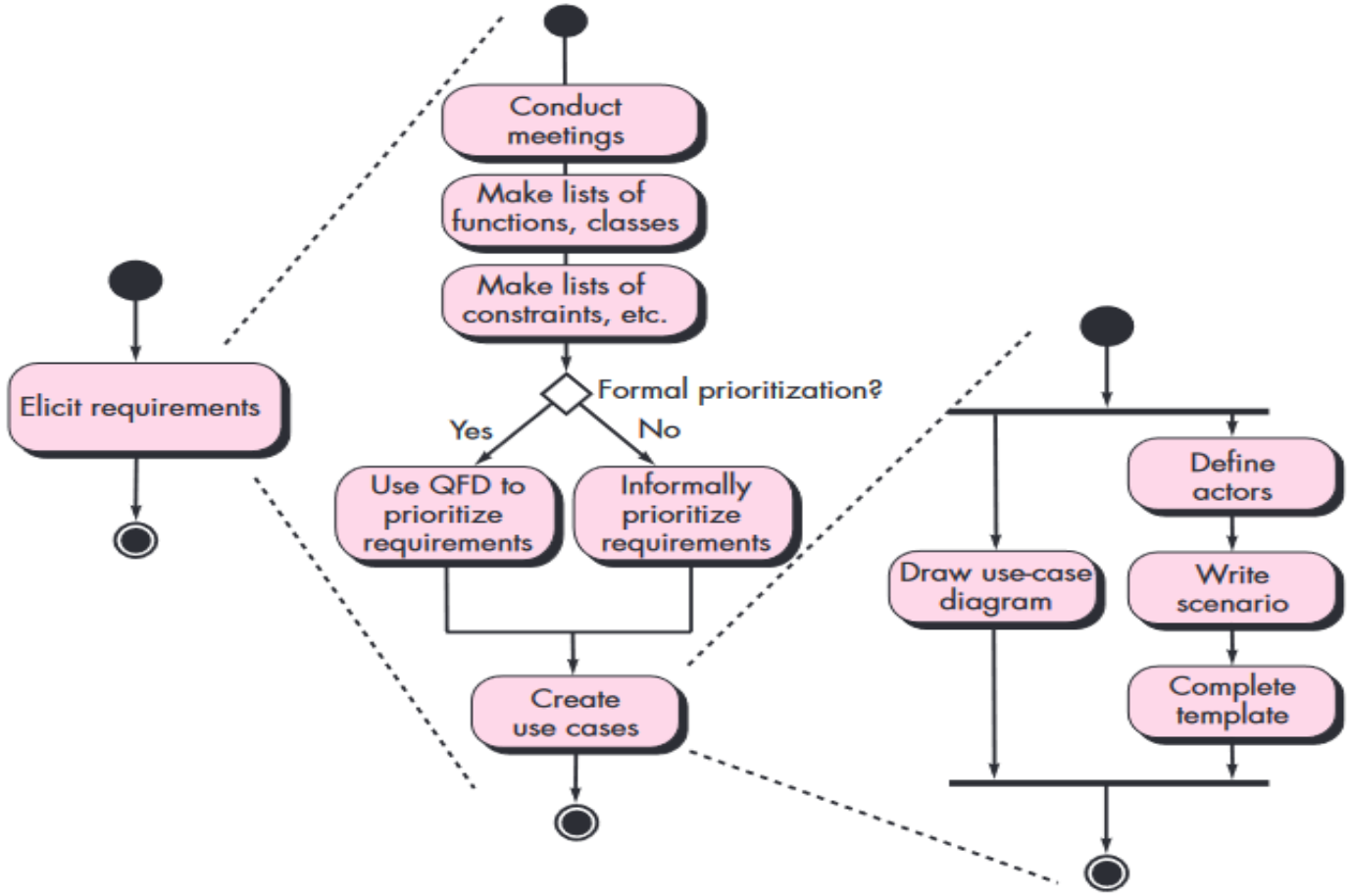
UML use case diagram for *SafeHome* home security function



UML ACTIVITY DIAGRAM FOR ELICITING REQUIREMENTS

FIGURE 5.3

UML activity diagrams for eliciting requirements



ELEMENTS OF THE REQUIREMENTS MODEL

- **CLASS-BASED ELEMENTS**

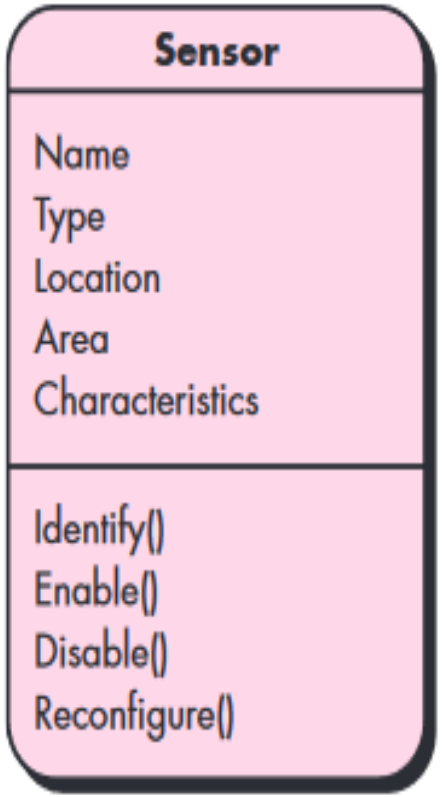
EACH USAGE SCENARIO IMPLIES A SET OF OBJECTS THAT ARE MANIPULATED AS AN ACTOR INTERACTS WITH THE SYSTEM. THESE OBJECTS ARE CATEGORIZED INTO CLASSES—A COLLECTION OF THINGS THAT HAVE SIMILAR ATTRIBUTES AND COMMON BEHAVIORS. **FOR EXAMPLE**, A UML CLASS DIAGRAM CAN BE USED TO DEPICT A **SENSOR** CLASS FOR THE SAFE HOME SECURITY FUNCTION (FIGURE 5.4).

NOTE THAT THE DIAGRAM LISTS THE **ATTRIBUTES OF SENSORS** (E.G., NAME, TYPE) AND THE OPERATIONS (E.G., IDENTIFY, ENABLE) THAT CAN BE APPLIED TO MODIFY THESE ATTRIBUTES

CLASS DIAGRAM FOR SENSORS

FIGURE 5.4

Class diagram for sensor



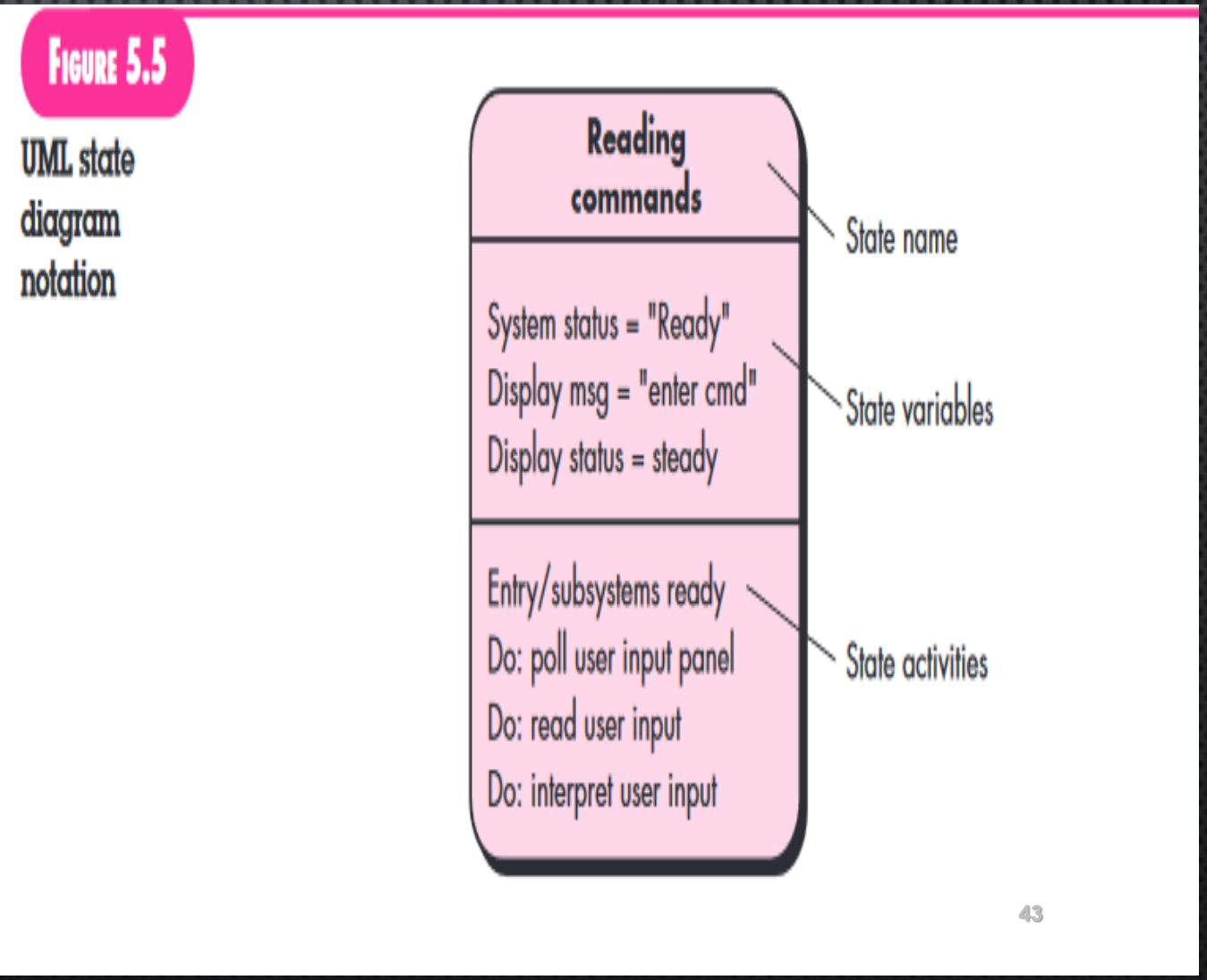
ELEMENTS OF THE REQUIREMENTS MODEL

- **BEHAVIORAL ELEMENTS.**

THE BEHAVIOR OF A COMPUTER-BASED SYSTEM CAN HAVE A PROFOUND EFFECT ON THE DESIGN THAT IS CHOSEN AND THE IMPLEMENTATION APPROACH THAT IS APPLIED. THEREFORE, THE REQUIREMENTS MODEL MUST PROVIDE MODELING ELEMENTS THAT DEPICT BEHAVIOR.

UML STATE DIAGRAM

- To illustrate the use of a state diagram, consider software embedded within the Safe Home control panel that is responsible for reading user input. A simplified UML state diagram is shown in Figure 5.5



- The **state diagram** is one method for representing the behavior of a system by depicting its states and the events that cause the system to change state. A state is any externally observable mode of behavior. In addition, the state diagram indicates actions (e.g., process activation) taken as a consequence of a particular event.

- **Flow-oriented elements:** Information is transformed as it flows through a computer-based system. The system accepts input in a **variety of forms, applies functions to transform it**, and **produces output in a variety of forms**. Input may be a control signal transmitted by a transducer, a series of numbers typed by a human operator, a packet of information transmitted on a network link, or a voluminous data file retrieved from secondary storage.

ELEMENTS OF THE REQUIREMENTS MODEL

- **Analysis Patterns**

These analysis patterns [Fow97] suggest solutions **(e.g., a class, a function, a behavior)** within the application domain that can be reused when modeling many applications.

First, analysis patterns speed up the development of abstract analysis models that capture the main requirements of the concrete problem by providing reusable analysis models **with examples as well as a description of advantages and limitations.**

Second, analysis patterns facilitate the transformation of the analysis model into a design model by suggesting design patterns and reliable solutions for common problems.

NEGOTIATING REQUIREMENT

- In an ideal requirements engineering context, **the inception, elicitation, and elaboration** tasks determine customer requirements in sufficient detail to proceed to subsequent software engineering activities.
- The best negotiations strive for a "win-win" result. That is, stakeholders win by getting the **system or product** that satisfies the majority of their needs and you (as a member of the software team) win by working to realistic and achievable budgets and deadlines.

NEGOTIATING REQUIREMENT

- BOEHM [BOE98] DEFINES A SET OF NEGOTIATION ACTIVITIES AT THE BEGINNING OF EACH SOFTWARE PROCESS ITERATION. **RATHER THAN A SINGLE CUSTOMER COMMUNICATION ACTIVITY, THE FOLLOWING ACTIVITIES ARE DEFINED:**

1. **IDENTIFICATION OF THE SYSTEM OR SUBSYSTEM'S KEY STAKEHOLDERS.**
2. **DETERMINATION OF THE STAKEHOLDERS' "WIN CONDITIONS."**
3. **NEGOTIATION OF THE STAKEHOLDERS' WIN CONDITIONS TO RECONCILE THEM INTO A SET OF WIN-WIN CONDITIONS FOR ALL CONCERNED (INCLUDING THE SOFTWARE TEAM).**

- SUCCESSFUL COMPLETION OF THESE INITIAL STEPS ACHIEVES A WIN-WIN RESULT, WHICH BECOMES THE KEY CRITERION FOR PROCEEDING TO SUBSEQUENT SOFTWARE ENGINEERING ACTIVITIES.

VALIDATING REQUIREMENTS

- It is examined for inconsistency, omissions, and ambiguity. The requirements represented by the model are prioritized by the stakeholders and grouped within requirements packages that will be implemented as software increments. **A review of the requirements model addresses the following questions.**
- Is each requirement consistent with the overall objectives for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?

VALIDATING REQUIREMENTS

- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?
- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function, and behavior of the system to be built?

MODULE-2 CONTINUATION….

- **Requirements Modeling Scenarios, Information and Analysis classes**: Requirement Analysis, Scenario based modeling, UML models that supplement the Use Case, Data modeling Concepts, Class-Based Modeling.

REQUIREMENTS MODELING SCENARIOS, INFORMATION AND ANALYSIS CLASSES

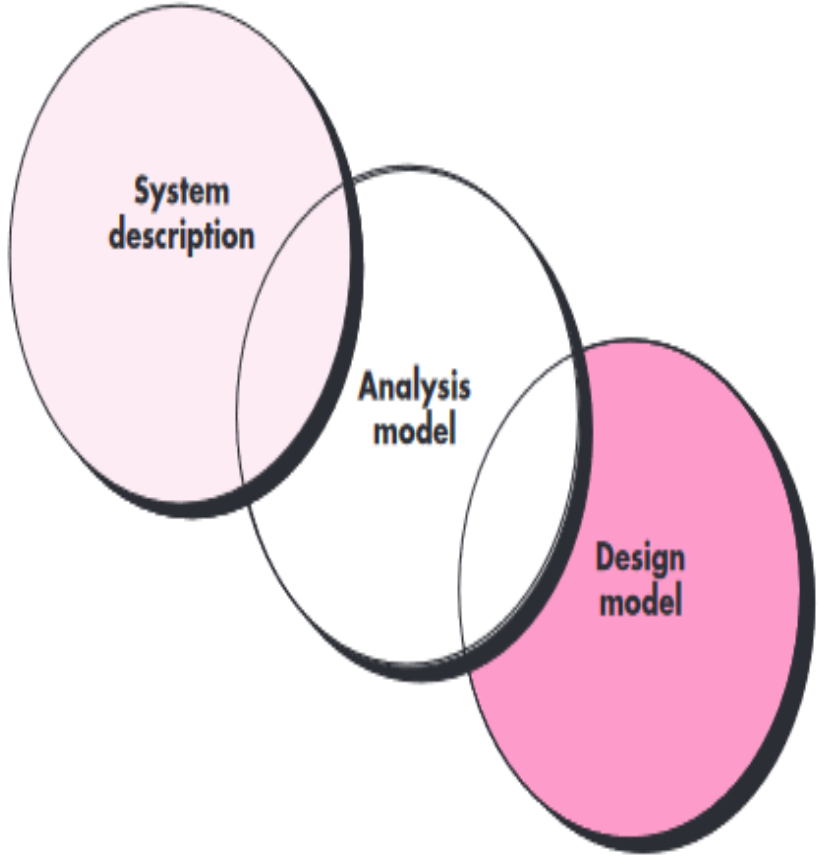
- **REQUIREMENT ANALYSIS**

REQUIREMENTS ANALYSIS RESULTS IN THE SPECIFICATION OF SOFTWARE'S **OPERATIONAL CHARACTERISTICS**, **INDICATES SOFTWARE'S INTERFACE WITH OTHER SYSTEM ELEMENTS**, AND **ESTABLISHES CONSTRAINTS THAT SOFTWARE MUST MEET.**

REQUIREMENTS ANALYSIS ALLOWS YOU (REGARDLESS OF WHETHER YOU'RE CALLED A SOFTWARE ENGINEER, AN ANALYST, OR A MODELER) TO ELABORATE ON BASIC REQUIREMENTS ESTABLISHED DURING THE INCEPTION, ELICITATION, AND NEGOTIATION TASKS THAT ARE PART OF REQUIREMENTS ENGINEERING.

FIGURE 6.1

The requirements model as a bridge between the system description and the design model



OVERALL OBJECTIVES AND PHILOSOPHY

- THE REQUIREMENTS MODEL MUST ACHIEVE THREE **PRIMARY OBJECTIVES**:

  (1) TO DESCRIBE WHAT THE CUSTOMER REQUIRES,

  (2) TO ESTABLISH A BASIS FOR THE CREATION OF A SOFTWARE DESIGN, AND

  (3) TO DEFINE A SET OF REQUIREMENTS THAT CAN BE VALIDATED ONCE THE SOFTWARE IS BUILT.

THE ANALYSIS MODEL BRIDGES THE GAP BETWEEN A SYSTEM LEVEL DESCRIPTION THAT DESCRIBES OVERALL SYSTEM OR BUSINESS FUNCTIONALITY AS IT IS ACHIEVED BY APPLYING SOFTWARE, HARDWARE, DATA, HUMAN, AND OTHER SYSTEM ELEMENTS AND A SOFTWARE DESIGN.

ANALYSIS RULES OF THUMB

- The model should focus on requirements that are visible within the problem or business **Don't get bogged down in details** domain. The level of abstraction should be relatively high. [Arl02] that try to explain how the system will work.
- Each element of the **requirements model** should add to an overall understanding of software requirements and provide insight into the information domain, function, and behavior of the system.
- Delay consideration of infrastructure and other **nonfunctional models** until design. That is, a database may be required, but the classes necessary to implement it, the functions required to access it, and the behavior that will be exhibited as it is used should be considered only after problem domain analysis has been completed.

ANALYSIS RULES OF THUMB

- Minimize coupling throughout the system. It is important to represent relation-ships between classes and functions. However, if the level of "**interconnectedness**" is extremely high, effort should be made to reduce it.
- Be certain that the requirements model provides value to all stakeholders. Each constituency has its own use for the model. For example, business stakeholders should use the model to validate requirements; designers should use the model as a basis for design; QA people should use the model to help plan acceptance tests.
- Keep the model as simple as it can be. Don't create additional diagrams when they add no new information. Don't use complex notational forms, when a simple list will do.

DOMAIN ANALYSIS

- The "**specific application domain**" can range from avionics to banking, from multimedia video games to software embedded within medical devices.
- **The goal of domain analysis is straightforward:** to find or create those analysis classes and/or analysis patterns that are broadly applicable so that they may be reused.

DOMAIN ANALYSIS

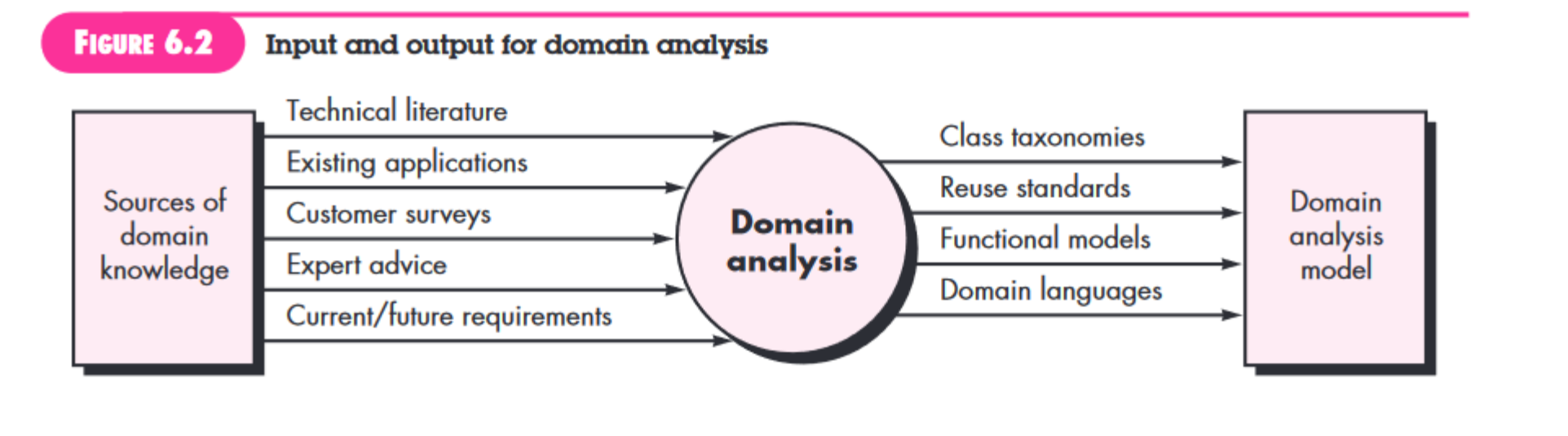


FIGURE 6.2 Input and output for domain analysis

The "**specific application domain**" can range from avionics to banking, from multimedia video games to software embedded within medical devices. The goal of domain analysis is straightforward: to find or create those analysis classes and/or analysis patterns that are broadly applicable so that they may be reused.

ELEMENTS OF ANALYSIS MODEL

Data and the processes that transform the data as separate entities. Data objects are modeled in a way that defines their attributes and relationships. Processes that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system.

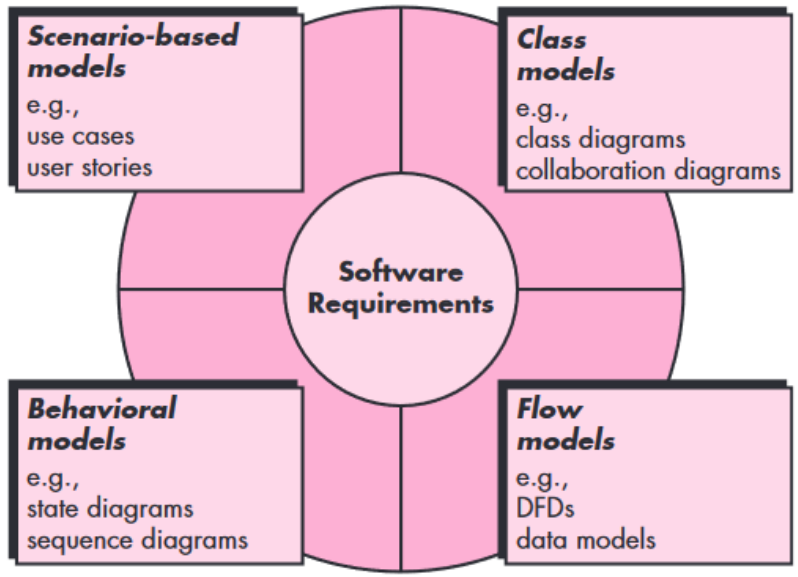
A second approach to analysis modeling, called object-oriented analysis, focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements. UML and the Unified Process are predominantly object oriented.

ELEMENTS OF ANALYSIS MODEL

FIGURE 6.3

Elements of the analysis model

What different points of view can be used to describe the requirements model?



SCENARIO BASED MODELING

- Creating a Preliminary Use Case the "**contract**" defines the way in which an actor uses a computer-based system to accomplish some goal. In essence, a use case captures the interactions that occur between producers and consumers of information and the system itself.

SCENARIO BASED MODELING

SCENARIO BASED MODELING HAS THE FOLLOWING STEPS

- CREATING A PRELIMINARY USE CASE
- REFINING A PRELIMINARY USE CASE
- WRITING A FORMAL USE CASE

CREATING A PRELIMINARY USE CASE

- To illustrate, consider the function access camera surveillance via the Internet—display camera views (ACS-DCV). The stakeholder who takes on the role of the homeowner actor might write the following narrative:
- **Use case: Access camera surveillance via the Internet—display camera views(please refer text book complete example)**

REFINING A PRELIMINARY USE CASE

- A description of alternative interactions is essential for a complete understanding of the function that is being described by a use case. Therefore, each step in the primary scenario is evaluated by asking the following questions [Sch98a]:
- Can the actor take some other action at this point?
- Is it possible that the actor will encounter some error condition at this point? If so, what might it be?
- Is it possible that the actor will encounter some other behavior at this point (e.g., behavior that is invoked by some event outside the actor's control)? If so, what might it be?

WRITING A FORMAL USE CASE

- **USE CASE: ACCESS CAMERA SURVEILLANCE VIA THE INTERNET—DISPLAY CAMERA VIEWS (ACS-DCV)**
- THE **ACS-DCV** USE CASE SHOWN IN THE SIDEBAR FOLLOWS A TYPICAL OUTLINE FOR FORMAL USE CASES. THE GOAL IN CONTEXT IDENTIFIES THE OVERALL SCOPE OF THE USE CASE. THE PRECONDITION DESCRIBES WHAT IS KNOWN TO BE TRUE BEFORE THE USE CASE IS INITIATED.
- THE TRIGGER IDENTIFIES THE EVENT OR CONDITION THAT "**GETS THE USE CASE STARTED**". THE SCENARIO LISTS THE SPECIFIC ACTIONS THAT ARE REQUIRED BY THE ACTOR AND THE APPROPRIATE SYSTEM RESPONSES.

UML MODELS THAT SUPPLEMENT THE USE CASE

- **UML MODELS THAT SUPPLEMENTS THE USE CASE ARE**
- ACTIVITY DIAGRAM
- SWIMLANE DIAGRAM
- **DEVELOPING AN ACTIVITY DIAGRAM**

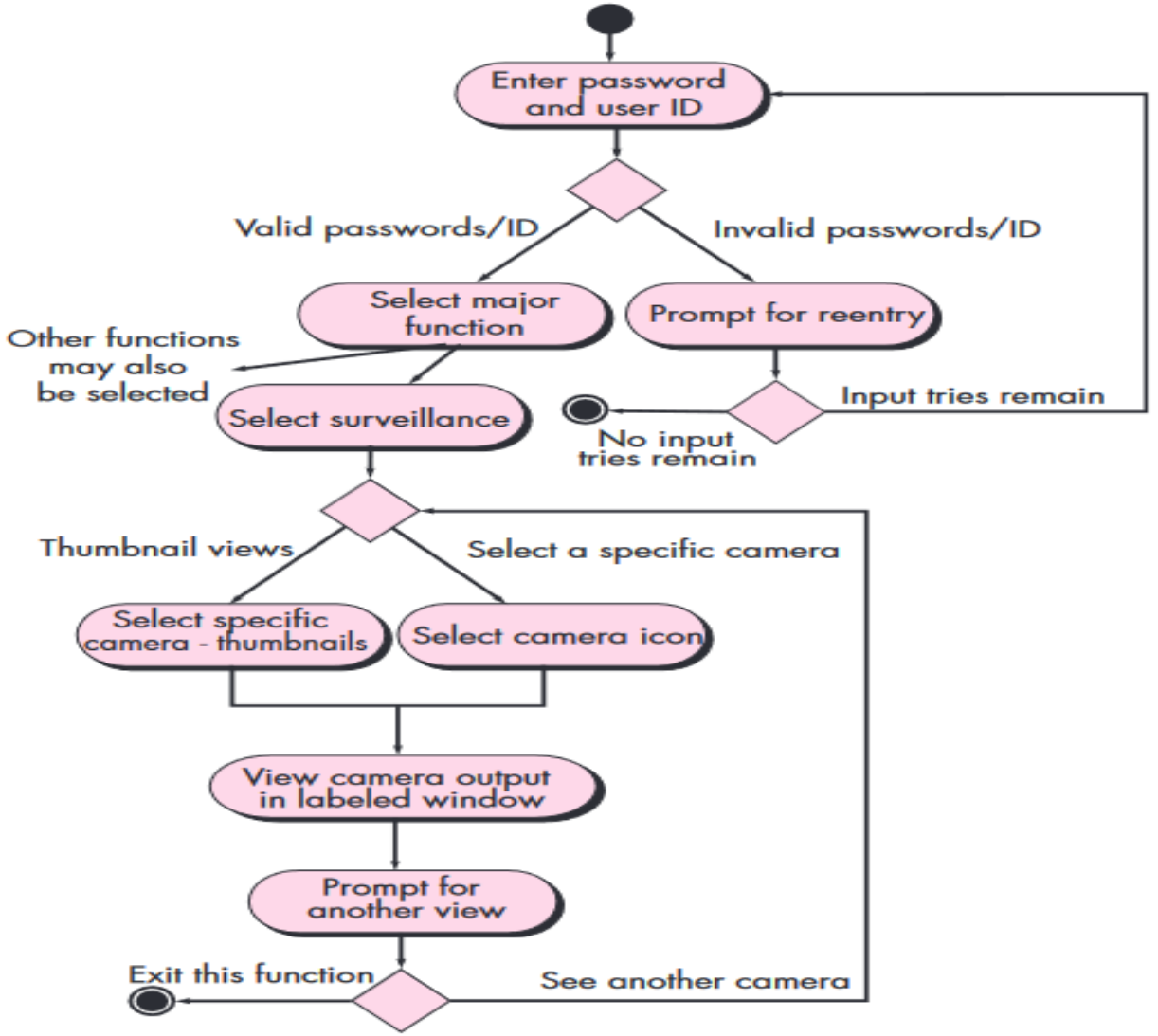
THE UML **ACTIVITY DIAGRAM** SUPPLEMENTS THE USE CASE BY PROVIDING A GRAPHICAL REPRESENTATION OF THE FLOW OF INTERACTION WITHIN A SPECIFIC SCENARIO.

SIMILAR TO THE FLOW CHART, AN **ACTIVITY DIAGRAM** USES ROUNDED RECTANGLES TO IMPLY A SPECIFIC SYSTEM FUNCTION, ARROWS TO REPRESENT FLOW THROUGH THE SYSTEM, DECISION DIAMONDS TO DEPICT A BRANCHING DECISION (EACH ARROW EMANATING FROM THE DIAMOND IS LABELED), AND SOLID HORIZONTAL LINES TO INDICATE THAT PARALLEL ACTIVITIES ARE OCCURRING.

ACTIVITY DIAGRAM FOR ACCESS CONTROL

FIGURE 6.5

Activity diagram for Access camera surveillance via the Internet—display camera views function.



SWIMLANE DIAGRAMS

The UML **swim lane diagram** is a useful variation of the activity diagram and allows you to represent the flow of activities described by the use case and at the same time indicate which actor (if there are multiple actors involved in a specific use case) or analysis class (discussed later in this chapter) has responsibility for the action described by an activity rectangle. Responsibilities are represented as parallel segments that divide the diagram vertically, like the lanes in a swimming pool.

Three analysis classes—Homeowner, Camera, and Interface—have direct or indirect responsibilities in the context of the activity diagram represented in Figure 6.5

SWIMLANE DIAGRAMS

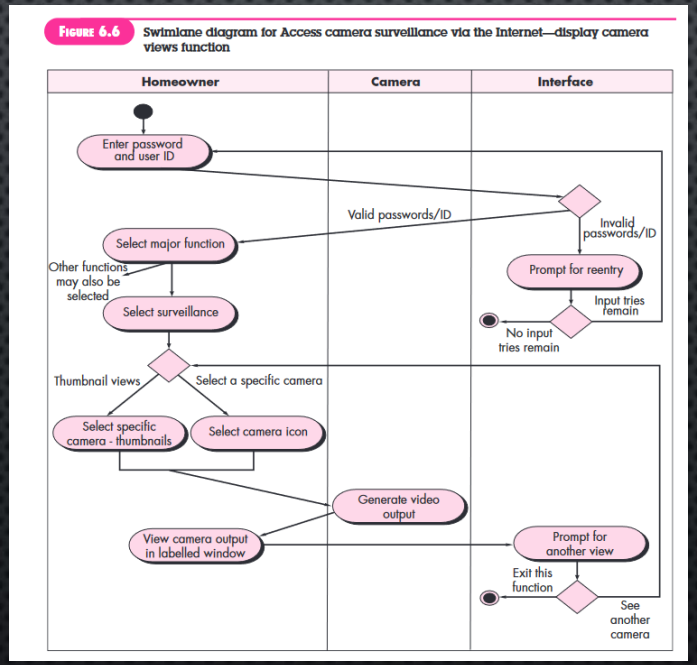


FIGURE 6.6 Swimlane diagram for Access camera surveillance via the Internet—display camera views function

THANK YOU